

Experiment 7: Breadth First Search (BFS) in Python

Aim

To write a Python program to implement the **Breadth First Search (BFS)** algorithm.

Algorithm

1. Create a graph using an adjacency list.
2. Start from the source node.
3. Mark the source node as visited.
4. Visit all adjacent unvisited nodes using a queue.
5. Repeat until the queue becomes empty.
6. Print the BFS traversal.

Program

```
from collections import deque
```

```
graph = {  
    'A': ['B', 'C'],  
    'B': ['D', 'E'],  
    'C': ['F'],  
    'D': [],  
    'E': [],  
    'F': []  
}
```

```
visited = set()
```

```
queue = deque(['A'])
```

```
print("BFS Traversal: ", end="")
```

```

while queue:
    node = queue.popleft()

    if node not in visited:
        print(node, end=" ")
        visited.add(node)

        for neighbour in graph[node]:
            if neighbour not in visited:
                queue.append(neighbour)

```

```
print("\n\nBFS Tree:")
```

```
print("    A")
```

```
print("  / \")
```

```
print(" B  C")
```

```
print(" / \  \")
```

```
print(" D  E  F")
```

Input

Graph:

A → B, C

B → D, E

C → F

D → None

E → None

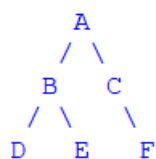
F → None

Starting Node = A

Output

```
BFS Traversal: A B C D E F
```

BFS Tree:



Result

Thus, the Python program to implement the Breadth First Search (BFS) algorithm was executed successfully, and the BFS traversal of the graph was displayed.